

Installation and Configuration of LAMP Stack on Ubuntu

LAMP Stack

A LAMP stack is a group of four open-source software components used to build dynamic websites and web applications. The term **LAMP** stands for:

- **L** - Linux (Operating System)
- **A** - Apache (Web Server)
- **M** – MySQL (Database Management System)
- **P** – PHP (Programming Language)

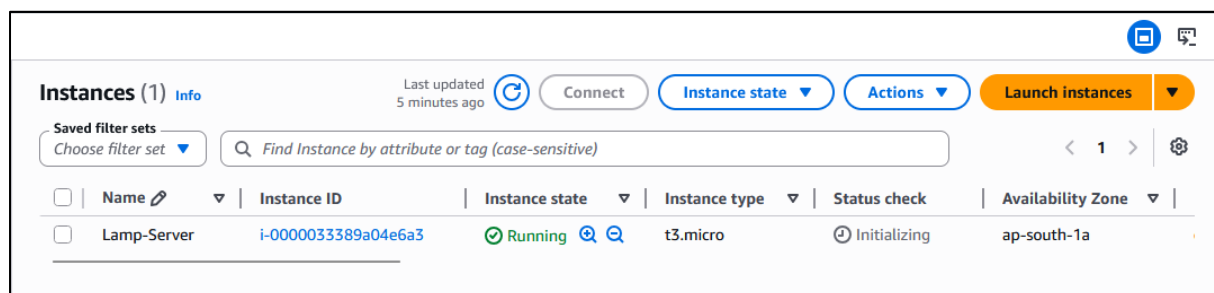
In a LAMP stack, Linux provides the operating environment, Apache handles web requests, MySQL stores and manages data, and PHP is used to create dynamic web pages. This stack is widely used because it is reliable, cost-effective, flexible, and easy to deploy for web development projects.

Step 1: Launch an Ubuntu Server in EC2

While launching the EC2 instance, choose the following options:

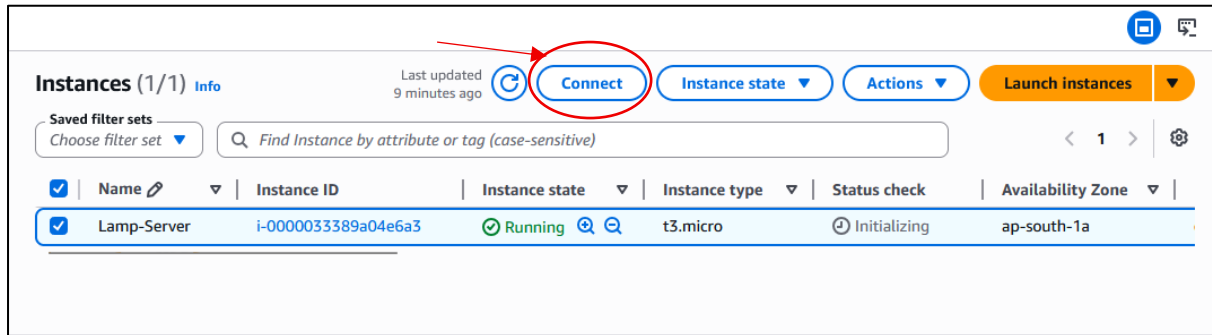
- **Name:** Give a name for your instance
- **AMI:** Select **Ubuntu Server (Latest LTS Version)**
- **Instance Type:** Choose **t2.micro** or **t3.micro**
- **Key Pair:** Create or select an existing key pair to connect via SSH
- **Network Settings:**
 - Allow **SSH (Port 22)**
 - Allow **HTTP (Port 80)**

Keep the remaining settings as default and launch the instance.



Step 2: Connect to EC2 instance

Choose the launched instance and click **Connect**.



Step 3: Install Apache Webserver

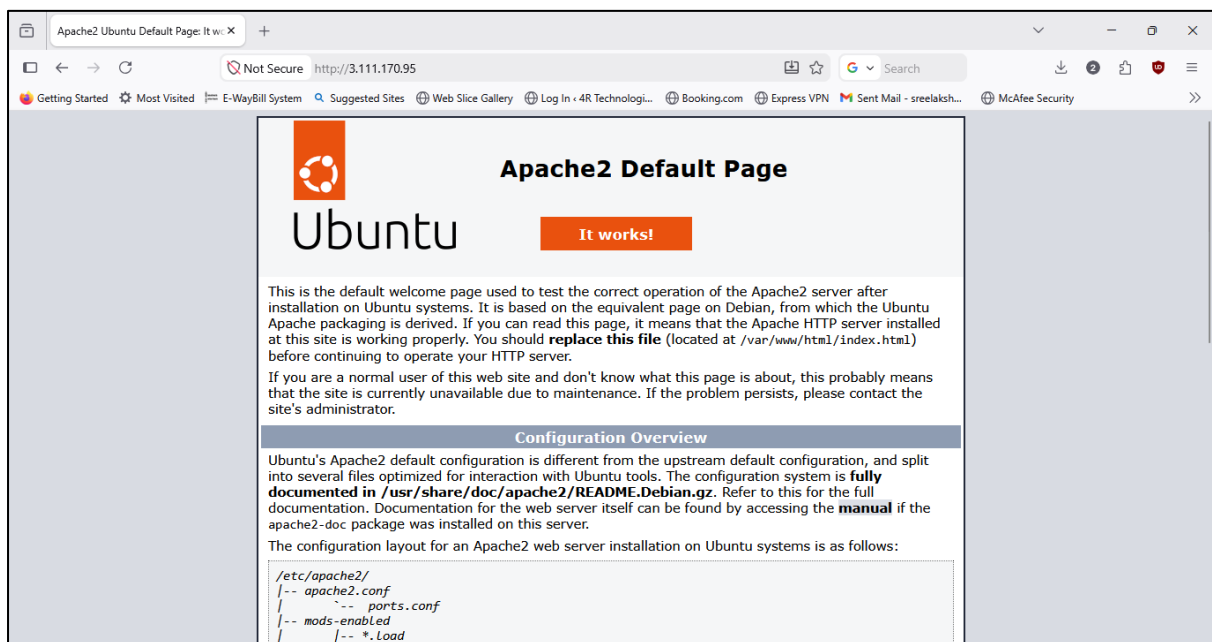
Update the package information index and upgrade your packages.

```
sudo apt update && sudo apt -y upgrade
```

Next, install the Apache Web server.

```
sudo apt install -y apache2
```

Paste the public IP address in the browser. If the installation was successful, the default Apache web page will appear.



Step 4: Install a Database Server

Run the following command to install MySQL server.

```
sudo apt install -y mysql-server
```

Run the below command to secure it.

```
sudo mysql_secure_installation
```

Enter the following choices:-

```
VALIDATE PASSWORD COMPONENT can be used to test passwords
and improve security. It checks the strength of password
and allows the users to set only those passwords which are
secure enough. Would you like to setup VALIDATE PASSWORD component?

Press y|Y for Yes, any other key for No: n
```

```
Remove anonymous users? (Press y|Y for Yes, any other key for No) : y
```

```
Disallow root login remotely? (Press y|Y for Yes, any other key for No) : y
```

```
Reload privilege tables now? (Press y|Y for Yes, any other key for No) : y
```

After securing installation, run the following command to login.

```
sudo mysql -u root -p
```

Enter the password and login.

Do the following set of commands on MySQL server:-

- Create a database

```
mysql> create database test_db;
Query OK, 1 row affected (0.02 sec)
```

- List the databases

```
mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
| test_db |
+-----+
5 rows in set (0.02 sec)
```

- Create a user

Replace 'Example_Password' with your own password.

```
mysql> create user 'test_user'@'localhost' identified by 'Example_Password';
```

- Grant full privileges to the database.

```
mysql> grant all privileges on test_db.* to 'test_user'@'localhost';  
Query OK, 0 rows affected (0.00 sec)
```

- Refresh the privilege settings.

```
mysql> flush privileges;  
Query OK, 0 rows affected (0.00 sec)
```

- Exit the MySQL server

```
mysql> exit;  
Bye
```

Step 5: Install PHP

Run the command below to install PHP.

```
sudo apt install -y php
```

Install the most common PHP extensions.

```
sudo apt install -y php-{common,mysql,xml,xmlrpc,curl,gd,imagick,cli,dev,imap,mbstring,opcache,soap,zip,intl}
```

Restart the Apache web server.

```
sudo systemctl restart apache2
```

Create an info.php file on the root directory of the web server.

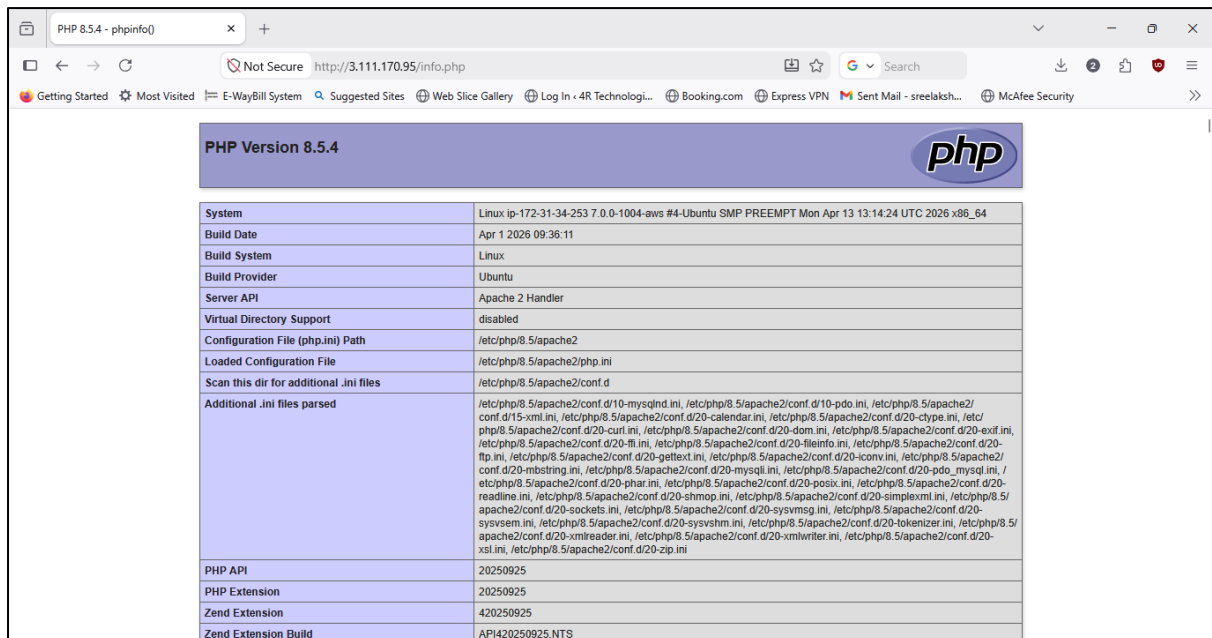
```
sudo nano /var/www/html/info.php
```

Enter the following in the file.

```
<?php  
phpinfo();
```

Save and exit by pressing 'Ctrl + x'.

Open the URL ‘http://your_public_ip/info.php’



PHP Version 8.5.4	
System	Linux ip-172-31-34-253 7.0.0-1004-aws #4-Ubuntu SMP PREEMPT Mon Apr 13 13:14:24 UTC 2026 x86_64
Build Date	Apr 1 2026 09:36:11
Build System	Linux
Build Provider	Ubuntu
Server API	Apache 2 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php/8.5/apache2
Loaded Configuration File	/etc/php/8.5/apache2/php.ini
Scan this dir for additional .ini files	/etc/php/8.5/apache2/conf.d
Additional .ini files parsed	/etc/php/8.5/apache2/conf.d/10-mysqld.ini, /etc/php/8.5/apache2/conf.d/10-pdo.ini, /etc/php/8.5/apache2/conf.d/15-xml.ini, /etc/php/8.5/apache2/conf.d/20-calendar.ini, /etc/php/8.5/apache2/conf.d/20-ctype.ini, /etc/php/8.5/apache2/conf.d/20-curl.ini, /etc/php/8.5/apache2/conf.d/20-dom.ini, /etc/php/8.5/apache2/conf.d/20-exif.ini, /etc/php/8.5/apache2/conf.d/20-fileinfo.ini, /etc/php/8.5/apache2/conf.d/20-ftp.ini, /etc/php/8.5/apache2/conf.d/20-gettext.ini, /etc/php/8.5/apache2/conf.d/20-iconv.ini, /etc/php/8.5/apache2/conf.d/20-ldap.ini, /etc/php/8.5/apache2/conf.d/20-mbstring.ini, /etc/php/8.5/apache2/conf.d/20-mysqli.ini, /etc/php/8.5/apache2/conf.d/20-pdo_mysql.ini, /etc/php/8.5/apache2/conf.d/20-phar.ini, /etc/php/8.5/apache2/conf.d/20-posix.ini, /etc/php/8.5/apache2/conf.d/20-readline.ini, /etc/php/8.5/apache2/conf.d/20-shmop.ini, /etc/php/8.5/apache2/conf.d/20-simplexml.ini, /etc/php/8.5/apache2/conf.d/20-sockets.ini, /etc/php/8.5/apache2/conf.d/20-sysvmsg.ini, /etc/php/8.5/apache2/conf.d/20-sysvshm.ini, /etc/php/8.5/apache2/conf.d/20-tokenizer.ini, /etc/php/8.5/apache2/conf.d/20-xmlreader.ini, /etc/php/8.5/apache2/conf.d/20-xmlwriter.ini, /etc/php/8.5/apache2/conf.d/20-xsl.ini, /etc/php/8.5/apache2/conf.d/20-zip.ini
PHP API	20250925
PHP Extension	20250925
Zend Extension	420250925
Zend Extension Build	API420250925.NTS

Test PHP connectivity with the database

```
sudo nano /var/www/html/database_test.php
```

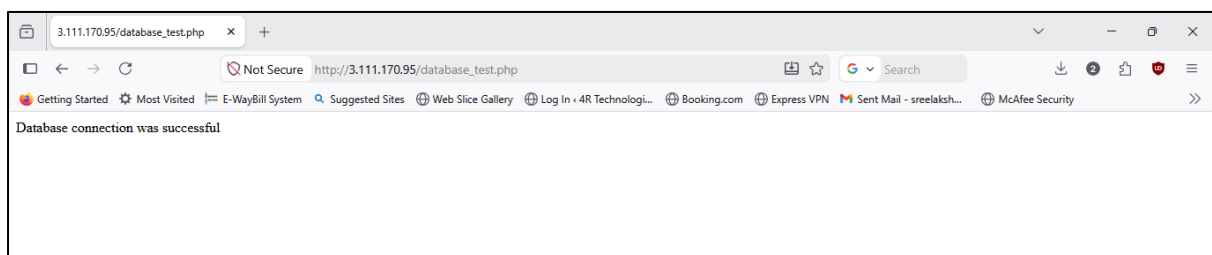
Enter the following in the file:-

Replace ‘Example_Password’ with your own password.

```
<?php
$conn = new mysqli('localhost', 'test_user', 'Example_Password', 'test_db');
if ($conn->connect_error) {
    die("Database connection failed: " . $conn->connect_error);
}
echo "Database connection was successful";
```

Save and exit by pressing ‘Ctrl + x’.

Open the URL ‘http://your_public_ip/database_test.php’



Conclusion

LAMP stack was successfully installed and configured on an Ubuntu EC2 instance in AWS. Apache was used as the web server, MySQL as the database server, and PHP as the scripting language. The setup process included launching and connecting to the EC2 instance, installing and configuring Apache, securing the MySQL server, creating a database and user, and integrating PHP with Apache. PHP connectivity with the MySQL database was also verified using a test PHP script. This implementation demonstrates how a LAMP stack can be deployed on a cloud-based Ubuntu server to host dynamic and database-driven web applications efficiently.